

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
(FACYT)
LICENCIATURA EN COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR - 4TO SEMESTRE

PRÁCTICA DE LABORATORIO 2

**Implementación y Análisis de Algoritmos de
Ordenamiento en MIPS32**

Estudiante:
Gabriel Vargas
C.I. 32.035.567

Docente:
Prof. José Canache

Naguanagua, Agosto 2026

Desarrollo del Informe

1. ¿Qué diferencias existen entre registros temporales (\$t0-\$t9) y registros guardados (\$s0-\$s7) y cómo se aplicó esta distinción en la práctica?

En la arquitectura MIPS32, los registros temporales (\$t0-\$t9) son aquellos que no preservan su valor a través de llamadas a funciones (son *caller-saved*). Si una función necesita su valor después de hacer un `jal`, debe respaldarlos en la pila. Por otro lado, los registros guardados (\$s0-\$s7) deben mantener su valor inalterado para la función que hizo la llamada (son *callee-saved*). Si una subrutina necesita usar un registro \$s, es su responsabilidad guardar su valor original en la pila al inicio (prólogo) y restaurarlo antes de retornar (epílogo). En esta práctica, en la implementación de Quicksort se preservó la convención de no alterar registros \$s sin guardarlos. Especialmente en el Mergesort Iterativo, se utilizaron múltiples registros \$s (\$s0-\$s5) para las variables de control de los bucles externos e internos (como `curr_size`, `left_start`, etc.), guardándolos estrictamente en la pila al comienzo de la función y restaurándolos al final, garantizando que cualquier función superior no perdiera sus datos.

2. ¿Qué diferencias existen entre los registros \$a0-\$a3, \$v0-\$v1, \$ra y cómo se aplicó esta distinción en la práctica?

Estos registros tienen roles fundamentales en el protocolo de llamadas a procedimientos:

- **\$a0-\$a3 (Argumentos):** Se utilizan para pasar los primeros cuatro argumentos a una función. En la práctica, \$a0 contuvo la dirección base del arreglo, mientras que \$a1, \$a2 y \$a3 se usaron para pasar índices (como `low`, `high`, `mid`).
- **\$v0-\$v1 (Valores de retorno):** Se usan para devolver resultados desde una función o para indicar códigos de llamadas al sistema (`syscall`). En la subrutina `partition` del Quicksort, se empleó \$v0 para retornar la posición final del pivote.
- **\$ra (Return Address):** Almacena la dirección de retorno luego de una instrucción `jal`. En algoritmos como Quicksort (recursivo) y Mergesort (que llama repetidamente a `merge`), fue indispensable respaldar \$ra en la pila (memoria) antes de hacer llamadas anidadas, ya que de lo contrario, el valor se sobrescribiría y el programa caería en un bucle infinito o fallaría al intentar retornar al `main`.

3. ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

El acceso a los registros del procesador es considerablemente más rápido (generalmente un ciclo de reloj) en comparación con el acceso a la memoria principal (instrucciones `lw` y `sw`), el cual puede tomar varios ciclos adicionales o introducir latencias si no hay aciertos en la caché. En los algoritmos implementados, maximizar el uso de registros para iteradores temporales y variables de cálculo intermedio mejoró el rendimiento. Sin embargo, al usar arreglos, el acceso a memoria es inevitable. En el caso del Mergesort, el uso de un arreglo temporal adicional en memoria para la fusión incrementa drásticamente la carga sobre el bus de datos en comparación con Quicksort, que realiza intercambios (*swaps*) *in-place*, demostrando ser más amigable con el rendimiento al minimizar la huella de memoria.

4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Las estructuras de control se traducen en instrucciones de salto condicional (`beq`, `bgt`, `ble`) e incondicional (`j`, `jal`). En el cauce (*pipeline*) de MIPS32, un salto puede provocar burbujas o paradas (*stalls*) si la condición no se evalúa a tiempo o si la predicción de salto falla. En el Mergesort Iterativo, que posee tres niveles lógicos de bucles anidados (`outer_loop`, `inner_loop`, y los bucles de `merge`), la cantidad de saltos es alta. Si no se organizan de forma óptima las condiciones de salida, el costo por penalización de saltos mal predichos reduce la eficiencia general de ejecución del procesador.

5. ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

Quicksort posee una complejidad promedio de $\mathcal{O}(n \log n)$, pero en el peor de los casos (arreglos ya ordenados o inversamente ordenados con un mal pivote) decae a $\mathcal{O}(n^2)$. Mergesort, por su parte, garantiza $\mathcal{O}(n \log n)$ en todos los casos. En el contexto de MIPS32 y un entorno limitado como MARS, Quicksort tiene la ventaja de ordenar *in-place* ($\mathcal{O}(\log n)$ de complejidad espacial debido a la pila recursiva), mientras que Mergesort requiere memoria auxiliar $\mathcal{O}(n)$. Esto implica que para arreglos muy grandes en MIPS32, Mergesort consumirá más espacio en el segmento de datos (*Data Segment*) y ejecutará muchas más instrucciones de copia a memoria, afectando el tiempo total de ciclos de máquina.

6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

El ciclo consta típicamente de cinco etapas (*pipeline* clásico RISC):

- a) **Fetch (IF):** Búsqueda de la instrucción en la memoria de instrucciones apuntada por el PC (Program Counter).

- b) **Decode (ID):** Decodificación de la instrucción y lectura simultánea de los registros fuente desde el banco de registros. También se realiza la extensión de signo de ser necesario.
- c) **Execute (EX):** La Unidad Aritmético Lógica (ALU) ejecuta la operación (suma, resta, cálculo de direcciones para arreglos, etc.).
- d) **Memory (MEM):** Acceso a la memoria de datos. Si es un `lw`, lee; si es `sw`, escribe. Las demás instrucciones pasan esta etapa sin hacer nada.
- e) **Write Back (WB):** El resultado de la ALU o el dato leído de memoria se escribe de vuelta en el registro de destino.

7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

Hubo una prevalencia importante de instrucciones tipo **I** y tipo **R**. Las instrucciones tipo **I** (`lw`, `sw`, `addi`, `beq`, `bgt`) fueron indispensables para la interacción con la memoria (arreglos y pila), ajustes de desplazamientos y el control del flujo (bucles). Las instrucciones tipo **R** (`add`, `sub`, `sll`, `move` - que se traduce a `addu`) se utilizaron para las operaciones aritméticas de cálculo de índices (multiplicar por 4 usando `sll`) y sumas lógicas. Las instrucciones tipo **J** (`j`, `jal`) se usaron específicamente para las llamadas a rutinas y retornos.

8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

El abuso de saltos impacta negativamente el rendimiento debido al concepto de *control hazards* (riesgos de control). Cada vez que ocurre un salto, el procesador debe alterar el flujo secuencial del Program Counter. Si se implementa en un procesador segmentado (*pipelined*), un salto que se toma repentinamente obliga a descartar las instrucciones que ya habían entrado al cauce (*flush*), generando un desperdicio de ciclos de reloj. Minimizar saltos reorganizando el código para que sea lo más secuencial posible (desenrollado de bucles, por ejemplo) aumenta sustancialmente el *throughput* (rendimiento por ciclo).

9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo RISC (Computación con Conjunto de Instrucciones Reducido) de MIPS ofrece una arquitectura ortogonal donde todas las instrucciones tienen un tamaño fijo de 32 bits, lo que simplifica la decodificación. Al separar strictly las operaciones aritméticas (que solo ocurren entre registros) de las de acceso a memoria (arquitectura *load/store*), obliga al programador a optimizar el uso de los 32 registros de propósito general. Para algoritmos de ordenamiento, esto resulta en un control

absoluto y muy fino sobre qué variables van a memoria y cuáles se mantienen en el procesador, permitiendo implementaciones altamente eficientes.

10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

El modo de ejecución paso a paso fue crucial durante la depuración lógica. Permitted aislar errores de segmentación (*segmentation faults*) causados por cálculos incorrectos de las direcciones de memoria. Al usar *Step Into*, se pudo descender a las subrutinas **partition** y **merge** y observar instrucción por instrucción cómo se desplazaban los punteros sobre los elementos del arreglo. Además, fue vital para confirmar que el puntero de pila (**\$sp**) regresaba exactamente a su valor original al terminar cada ciclo recursivo o iterativo, garantizando que no existieran fugas de memoria en la pila.

11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

Las ventanas laterales del entorno MARS, específicamente el panel de **Registers** y el panel de **Data Segment**, fueron las herramientas más útiles. El panel de registros permitió monitorear en tiempo real que variables como los índices (**low**, **high**, **mid**) tomaran los valores matemáticamente correctos. El Data Segment permitió visualizar el mapa de memoria donde estaba alojado el arreglo, pudiendo confirmar visualmente el intercambio de números hexadecimales a medida que los algoritmos hacían los *swaps* y ordenaban el vector.

12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

Aunque MARS no provee una animación esquemática nativa profunda del datapath completo del hardware, el comportamiento del camino de datos tipo R se deduce analizando los cambios de estado. Para un **add \$t0, \$t1, \$t2**, se lee el código máquina generado en la ventana *Execute*. Internamente: el Opcode es 0, **rs** lee \$t1, **rt** lee \$t2. Las señales de control activan **RegDst = 1** (destino es **rd** que sería \$t0), **ALUSrc = 0** (el segundo operando viene del registro, no del inmediato), y **RegWrite = 1**. El resultado calculado en la ALU fluye por los multiplexores directamente al puerto de escritura del banco de registros, evadiendo la memoria de datos (**MemWrite = 0**, **MemRead = 0**).

13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

Para una instrucción como **lw \$t0, 0(\$t1)**, el comportamiento difiere en el camino de datos: la instrucción extrae el operando **rs** (\$t1) y lo envía a la ALU. El valor inmediato (0) pasa por una unidad de extensión de signo (de 16 a 32 bits). La señal

de control `ALUSrc = 1` asegura que este inmediato extienda el segundo puerto de la ALU. La ALU suma ambos valores para obtener la dirección efectiva en memoria. Luego, las señales en la etapa de memoria se activan: `MemRead = 1`. Finalmente, un multiplexor controlado por `MemoReg = 1` dirige el valor leído de la RAM hacia el registro de destino `rt ($t0)`, con `RegWrite = 1` y `RegDst = 0`.

14. Justificar la elección del algoritmo alternativo

Como alternativa a la implementación recursiva inicial, se eligió implementar **Mergesort de forma iterativa (Bottom-Up)**. Esta decisión se tomó para establecer un contraste técnico directo frente al Quicksort recursivo. Mientras que la recursión impone una gran carga cognitiva y computacional en la gestión de la memoria de pila (por los guardados continuos de `$ra` y argumentos), el Mergesort iterativo anula la necesidad del crecimiento del *stack frame* en llamadas múltiples. Esto resultó en un excelente ejercicio para manejar registros estables (`$s0-$s7`) y dominar la anidación de bucles (tres ciclos anidados), explorando a profundidad estructuras de control en lenguaje ensamblador puro sin el amparo del entorno recursivo estándar.

15. Análisis y Discusión de los Resultados

La traducción de algoritmos de alto nivel como Quicksort y Mergesort a lenguaje ensamblador MIPS32 evidenció el abismo existente entre la abstracción de lenguajes como C o Python y el manejo a nivel de máquina.

En primer lugar, los resultados demostraron que Quicksort, pese a ser recursivo, ofrece un rendimiento percibido muy alto debido a que trabaja el ordenamiento de los elementos *in-place*. Su gestión de la pila, aunque requiere cuidado aritmético estricto (restar múltiplos de 4 al apuntador `$sp`), resultó limpia.

Por otro lado, Mergesort (en su variante Bottom-Up) demostró ser impecable en su enfoque iterativo, pero introdujo la dificultad de gestionar la memoria del segmento de datos al requerir un bloque auxiliar de palabras (`temp`) para realizar la fusión. Esto incrementó exponencialmente el uso de instrucciones de lectura y escritura (`lw/sw`).

En conclusión, la práctica cumplió el objetivo de asentar firmemente la mecánica del ciclo de ejecución RISC. Se comprobó que el control riguroso de los punteros multiplicados por 4 (para alinear a palabra de 32 bits) y la correcta distinción entre registros volátiles y no volátiles son la clave del éxito en el diseño arquitectónico de software a bajo nivel.